Name: Shruti Biradar

PRN No: 202402040027

Group Member:
Saishwari Korade : 202402040028

# LSTM-Based Sequence Prediction System with FastAPI Deployment

## 1. Introduction

Sequence prediction is an important problem in artificial intelligence and natural language processing. It involves predicting the next element in a sequence based on previously observed elements. In natural language processing, this often refers to predicting the next word in a sentence.

In this project, we developed an AI-based sequence prediction system using Long ShortTerm Memory (LSTM) networks. LSTM is a type of recurrent neural network (RNN) capable of learning long-term dependencies in sequential data.

The system was trained on a text dataset and deployed using FastAPI to provide real-time prediction capabilities through an API. The project demonstrates the complete pipeline of dataset collection, preprocessing, model training, evaluation, and deployment.

## 2. Objective

The main objectives of this assignment are:

- To develop an LSTM-based sequence prediction model
- To preprocess and prepare text data for machine learning
- To train a deep learning model capable of predicting the next word
- To deploy the trained model using FastAPI for real-time inference
- To build an end-to-end AI system combining machine learning and API deployment

## 3. Dataset Used

Dataset Name: Tiny Shakespeare Dataset

Dataset Source:
https://raw.githubusercontent.com/karpathy/charrnn/master/data/tinyshakespeare/input.txt

Dataset Description:
The Tiny Shakespeare dataset contains dialogues from Shakespeare's plays. It provides thousands of lines of natural language text that allow deep learning models to learn sequential language patterns.

Example:

First Citizen:
Before we proceed any further, hear me speak.
All:
Speak, speak.

## 4. Data Preprocessing

Text preprocessing was performed before training the model.

Steps included:

Text Cleaning – Converting text to lowercase and removing punctuation.

Sentence Extraction – Splitting the dataset into individual sentences.

Tokenization – Converting words into numerical indices.

Sequence Generation – Creating input-output training sequences.

Padding – Making all sequences equal length so they can be processed by the neural network.

## 5. LSTM Model Architecture

The deep learning model consists of the following layers:

Embedding Layer – Converts word indices into dense vectors.

LSTM Layer – Learns sequential dependencies in the text.

Dropout Layer – Prevents overfitting.

Second LSTM Layer – Improves sequence learning.

Dense Layer – Predicts the probability of the next word.

Architecture:

Embedding → LSTM → Dropout → LSTM → Dense

## 6. Mathematical Model of LSTM

LSTM networks use gates to control information flow.

Forget Gate:
$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$

Input Gate:
$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$

Candidate Memory:
$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$

Cell State Update:
$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$

Output Gate:
$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$

Hidden State: ht
= ot * tanh(Ct)

## 7. Model Training

Training configuration:

Loss Function: Sparse Categorical Crossentropy
Optimizer: Adam Optimizer
Batch Size: 64
Epochs: 40

Sparse categorical crossentropy was used because labels were integer encoded rather than one-hot encoded.

## 8. Model Performance

Training Accuracy: Approximately 0.4

Accuracy increased gradually over epochs, demonstrating effective learning of language patterns.

Train Model

```
history = model.fit(
    X,
    y,
    epochs=40,
    batch_size=64
)
```

```
Epoch 12/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step - accuracy: 0.1375 - loss: 4.9310
Epoch 13/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step - accuracy: 0.1390 - loss: 4.8855
Epoch 14/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 7ms/step - accuracy: 0.1417 - loss: 4.8428
Epoch 15/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step - accuracy: 0.1438 - loss: 4.8009
Epoch 16/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 7ms/step - accuracy: 0.1456 - loss: 4.7629
Epoch 17/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step - accuracy: 0.1480 - loss: 4.7254
Epoch 18/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step - accuracy: 0.1487 - loss: 4.6897
Epoch 19/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 7ms/step - accuracy: 0.1512 - loss: 4.6560
Epoch 20/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step - accuracy: 0.1542 - loss: 4.6226
Epoch 21/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 7ms/step - accuracy: 0.1551 - loss: 4.5918
Epoch 22/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step - accuracy: 0.1576 - loss: 4.5614
Epoch 23/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step - accuracy: 0.1592 - loss: 4.5316
Epoch 24/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 7ms/step - accuracy: 0.1614 - loss: 4.5032
Epoch 25/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step - accuracy: 0.1637 - loss: 4.4757
Epoch 26/40
2673/2673 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step - accuracy: 0.1649 - loss: 4.4521
```

Loss decreased steadily during training, indicating improved model performance.

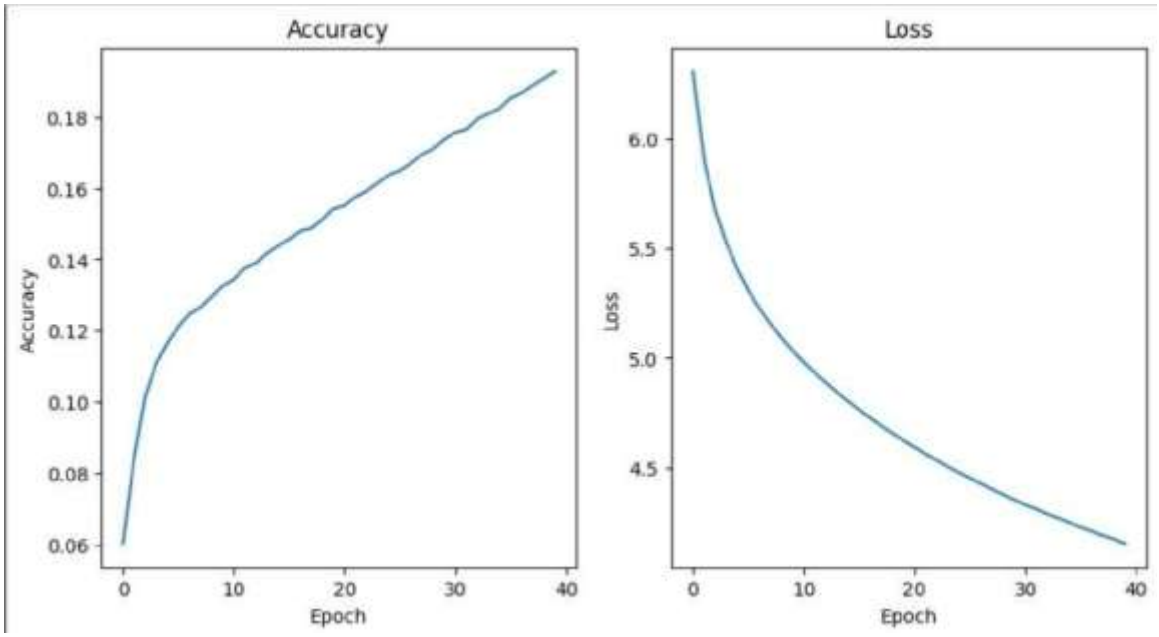
## 9. Prediction Results

```
print(generate_text("my lord",5))
print(generate_text("i will",5))
print(generate_text("the king",5))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step
my lord northumberland i find the way
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
i will and stay the king be
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
the king is <OOV> and made him
```

These predictions demonstrate that the LSTM model successfully learned sequence patterns.

## 10. Accuracy and Loss

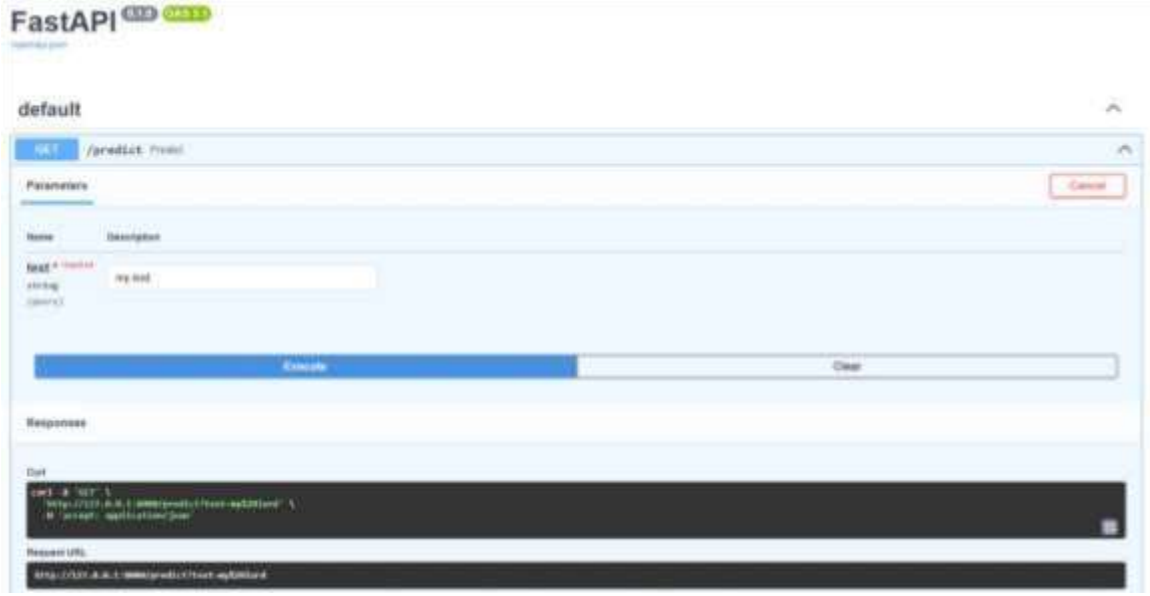



## 10. API Deployment Using FastAPI

The trained model was deployed using FastAPI.

API Endpoints:

/predict – Predict the next word
/generate – Generate multiple words



The API was tested using the Swagger interface at:

http://127.0.0.1:8000/docs

## 11. System Architecture

Dataset

↓

Data Preprocessing

↓

Tokenization
↓
Sequence Generation
↓
LSTM Model Training
↓
Prediction
↓
FastAPI Deployment
↓
Real-Time API Response

## 12. Applications

Sequence prediction systems are used in:

- Chatbots
- Text autocomplete systems
- Language translation
- AI assistants
- Text generation systems

## 13. Conclusion

This project successfully implemented an LSTM-based sequence prediction system. The model was trained on text data to learn language patterns and predict the next word in a sequence.

The trained model was deployed using FastAPI, enabling real-time predictions through an API. This project demonstrates the integration of deep learning and web technologies to build an industry-relevant AI application.